

Paimon Service 模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-service

分支：release-1.3.1

生成工具：AI Architecture Analyzer

生成日期：2026年03月27日

第1章 模块概述

1.1 模块定位

paimon-service 是 Apache Paimon 的分布式远程 KV 查询服务模块，为主键表（Primary Key Table）提供基于 Netty 的高性能远程 Lookup 查询能力。该模块采用经典的 Client-Server 架构，客户端根据 partition/bucket 信息定位目标服务器，发送 KV 查询请求，服务端在本地执行 TableQuery.lookup 操作并返回结果。

该模块由两个子模块组成：paimon-service-client（客户端库，包含网络通信框架和消息协议）和 paimon-service-runtime（服务端运行时，包含查询处理逻辑）。

1.2 核心目标

- 远程 KV 查询：支持客户端通过网络远程查询主键表中指定 partition/bucket 下的 key-value 数据
- 高性能网络通信：基于 Netty 实现异步、非阻塞的网络通信框架，支持连接池复用和分块传输
- 容错与自动重试：在 partition/bucket 路由错误或连接异常时自动重试，并强制刷新服务发现信息
- 可观测性：提供完善的请求统计（连接数、请求数、成功/失败数、延迟）机制

1.3 子模块结构

子模块	包路径	核心职责
paimon-service-client	o.a.p.service.client/messages/network	Netty 客户端、消息协议、序列化框架、连接管理
paimon-service-runtime	o.a.p.service.server	KV 查询服务器、请求处理器

1.4 源码文件清单

文件名	包	职责描述
KvQueryClient.java	client	KV 查询客户端，定位服务器并发送请求，支持失败重试
KvRequest.java	messages	KV 查询请求消息体（partition + bucket + keys）
KvResponse.java	messages	KV 查询响应消息体（values 数组，支持 null）
NetworkClient.java	network	基于 Netty 的通用网络客户端，管理连接池
NetworkServer.java	network	基于 Netty 的通用网络服务端，支持端口范围绑定

文件名	包	职责描述
AbstractServerHandler.java	network	服务端请求处理器基类，异步线程池处理请求
ClientHandler.java	network	客户端 Channel Handler，分发响应回调
ServerConnection.java	network	连接管理（Pending/Established 状态机）
MessageSerializer.java	network.messages	核心序列化器，定义帧格式和消息类型
MessageBody.java	network.messages	消息体抽象基类
MessageDeserializer.java	network.messages	消息反序列化接口
MessageType.java	network.messages	消息类型枚举（REQUEST/RESULT/FAILURE）
RequestFailure.java	network.messages	请求失败消息体
ChunkedByteBuf.java	network	大消息分块传输，配合 ChunkedWriteHandler
NettyBufferPool.java	network	Netty 内存池封装，仅使用 Direct Buffer
NetworkUtils.java	network	网络工具类，查找本地可用 IP 地址
ClientHandlerCallback.java	network	客户端回调接口
BadRequestException.java	network	错误请求基类异常
UnknownPartitionBucketException.java	exceptions	分区/桶不匹配异常
ServiceRequestStats.java	network.stats	统计接口
AtomicServiceRequestStats.java	network.stats	线程安全的统计实现
DisabledServiceRequestStats.java	network.stats	空操作统计实现
KvQueryServer.java	server	KV 查询服务器，继承 NetworkServer
KvServerHandler.java	server	KV 请求处理器，校验归属并执行 lookup

第2章 核心架构设计

2.1 整体分层架构

paimon-service 模块采用经典的四层架构设计，从上到下分别是：业务接口层、消息协议层、网络传输层、基础设施层。各层之间通过清晰的接口隔离，实现了高内聚低耦合的设计目标。

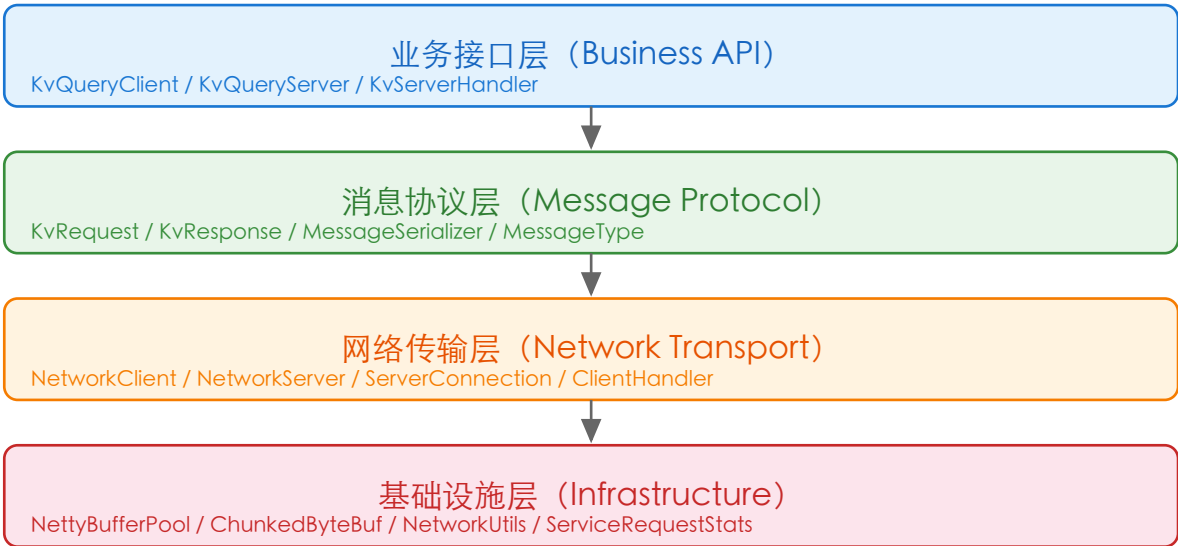


图2-1 paimon-service 整体分层架构图

2.2 客户端-服务端交互架构

客户端 KvQueryClient 通过 QueryLocation 服务发现定位目标服务器地址，然后通过 NetworkClient 建立 Netty 连接并发送 KvRequest。服务端 KvQueryServer 监听端口，KvServerHandler 接收请求后先验证 partition/bucket 归属，再调用 TableQuery.lookup 执行本地查询，最终返回 KvResponse。

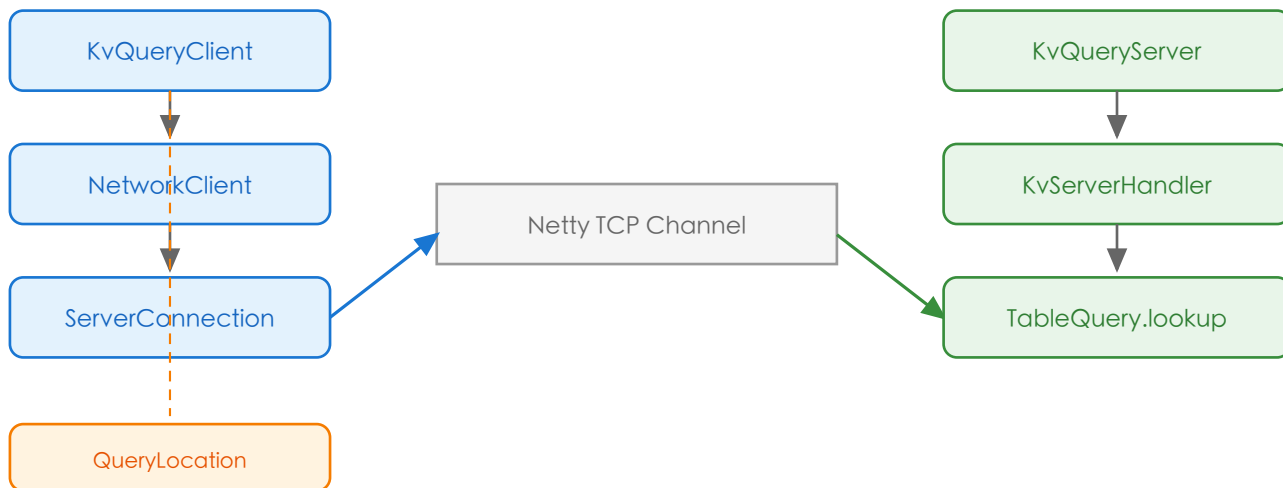


图2-2 客户端-服务端交互架构图

2.3 消息帧格式

所有网络消息遵循统一的二进制帧格式，由 MessageSerializer 定义。帧结构如下：

```

+-----+
| FRAME LENGTH (4) | VERSION (4) | TYPE (4) | CONTENT (VAR) |
+-----+
VERSION = 1, TYPE = REQUEST(0) / REQUEST_RESULT(1) / REQUEST_FAILURE(2) /
SERVER_FAILURE(3)
  
```

帧长度字段（4字节）由 Netty 的 LengthFieldBasedFrameDecoder 自动处理。VERSION 用于协议版本兼容性校验，TYPE 表示消息类型。

2.4 连接状态机

ServerConnection 内部使用 状态机模式 管理连接生命周期，定义了两种状态：

- PendingConnection：连接建立中，请求会排队等待。连接成功后批量转发排队请求到 EstablishedConnection
- EstablishedConnection：连接已建立，直接发送请求。实现 ClientHandlerCallback 处理响应回调

第3章 核心类设计模式详解

3.1 NetworkServer / NetworkClient — 模板方法 (Template Method) 模式

NetworkServer

是一个抽象基类，定义了服务器启动、端口绑定、关闭的完整生命周期模板。子类只需实现 `initializeHandler()` 抽象方法来提供具体的请求处理器，无需关心配置、端口冲突处理等底层细节。

Netty

关键方法：

- `start()` — 模板方法，遍历端口范围调用 `attemptToBind()` 尝试绑定
- `attemptToBind(port)` — 创建线程池、初始化 handler、配置 `ServerBootstrap` 并绑定端口
- `initializeHandler()` — 抽象方法，由子类 `KvQueryServer` 实现，返回 `KvServerHandler`
- `shutdownServer()` — 模板方法，协调关闭 `EventLoopGroup`、`Handler` 和查询线程池

同样地，`NetworkClient` 定义了客户端的连接管理和关闭模板。`KvQueryClient` 组合使用 `NetworkClient` 并添加重试逻辑。

3.2 AbstractServerHandler — 策略 (Strategy) + 异步命令 (Command) 模式

`AbstractServerHandler` 将请求处理逻辑抽象为 `handleRequest()` 方法，不同的 `Handler` 实现（如 `KvServerHandler`）可以插入不同的业务逻辑。`channelRead()` 中使用 `AsyncRequestTask` 将请求封装为异步命令提交到线程池执行，避免阻塞 Netty IO 线程。

关键设计：

- `handleRequest(requestId, request)` — 策略方法，返回 `CompletableFuture` 支持异步处理
- `AsyncRequestTask` — 内部类，封装为 `Runnable` 提交到 `queryExecutor` 线程池
- `RequestWriteListener` — 写入回调，记录成功/失败统计和延迟
- 大响应通过 `ChunkedByteBuf` 分块传输，遵循 Netty 高低水位线机制

3.3 ServerConnection — 状态 (State) 模式

`ServerConnection` 使用 `InternalConnection` 接口实现状态模式，在 `PendingConnection` 和 `EstablishedConnection` 之间切换：

- `PendingConnection`：连接建立中，将请求加入 `ArrayDeque` 排队。连接成功后创建 `EstablishedConnection` 并逐个转发排队请求
- `EstablishedConnection`：连接已就绪，直接序列化请求写入 `Channel`。维护 `pendingRequests ConcurrentHashMap` 跟踪请求-响应映射

状态切换通过 `establishConnection(ChannelFuture)` 方法触发，整个过程通过 `connectionLock` 保证线程安全。

3.4 MessageSerializer — 工厂方法（Factory Method）模式

MessageSerializer 通过组合 MessageDeserializer 接口实现工厂方法模式。不同的消息类型（KvRequest、KvResponse）各自提供内部 Deserializer 实现类。序列化器提供统一的静态工厂方法：

- serializeRequest() — 序列化请求消息
- serializeResponse() — 序列化响应消息
- serializeRequestFailure() — 序列化请求失败消息（Java 序列化异常对象）
- serializeServerFailure() — 序列化服务端失败消息

3.5 ServiceRequestStats — 策略（Strategy）模式

ServiceRequestStats 接口定义了统计行为，提供两个实现：

- AtomicServiceRequestStats：使用 AtomicLong 实现线程安全的统计收集
- DisabledServiceRequestStats：空操作实现（Null Object 模式），所有方法为空，用于不需要统计的场景

3.6 NettyBufferPool — 装饰器（Decorator）模式

NettyBufferPool 实现了 Netty 的 ByteBufferAllocator 接口，内部委托给 PooledByteBufferAllocator，但严格禁止堆内存分配——所有 heapBuffer() 方法都抛出 UnsupportedOperationException。这是装饰器模式的一种变体，通过限制父类行为来强制使用 Direct Buffer，减少 GC 压力。

第4章 核心流程图

4.1 客户端 KV 查询流程

客户端通过 `KvQueryClient.getValues()` 发起查询请求，内部通过 `QueryLocation` 定位服务器地址，`NetworkClient` 管理 TCP 连接并发送序列化请求。遇到 `UnknownPartitionBucketException` 或 `ConnectException` 时自动重试并强制刷新 location。

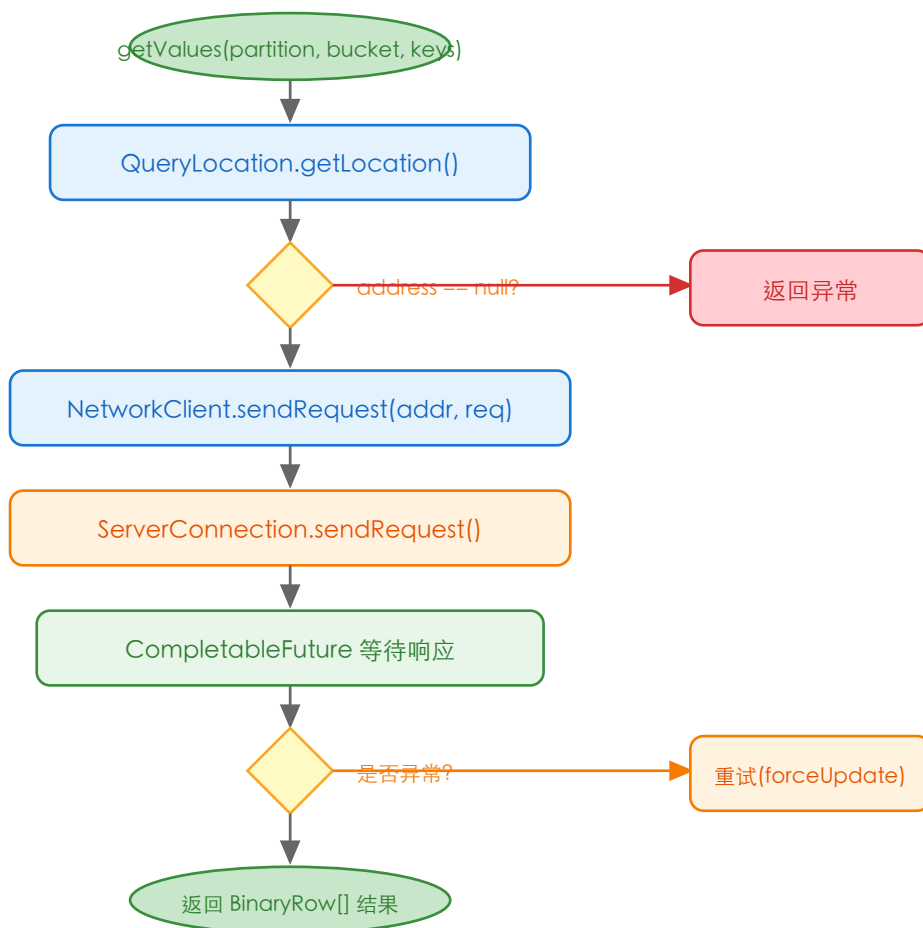


图4-1 客户端 KV 查询流程图

4.2 服务端请求处理流程

服务端通过 `AbstractServerHandler.channelRead()` 接收消息，反序列化后提交到线程池异步处理。`KvServerHandler.handleRequest()` 先校验 `partition/bucket` 归属，再逐个 `key` 调用 `TableQuery.lookup` 查询。

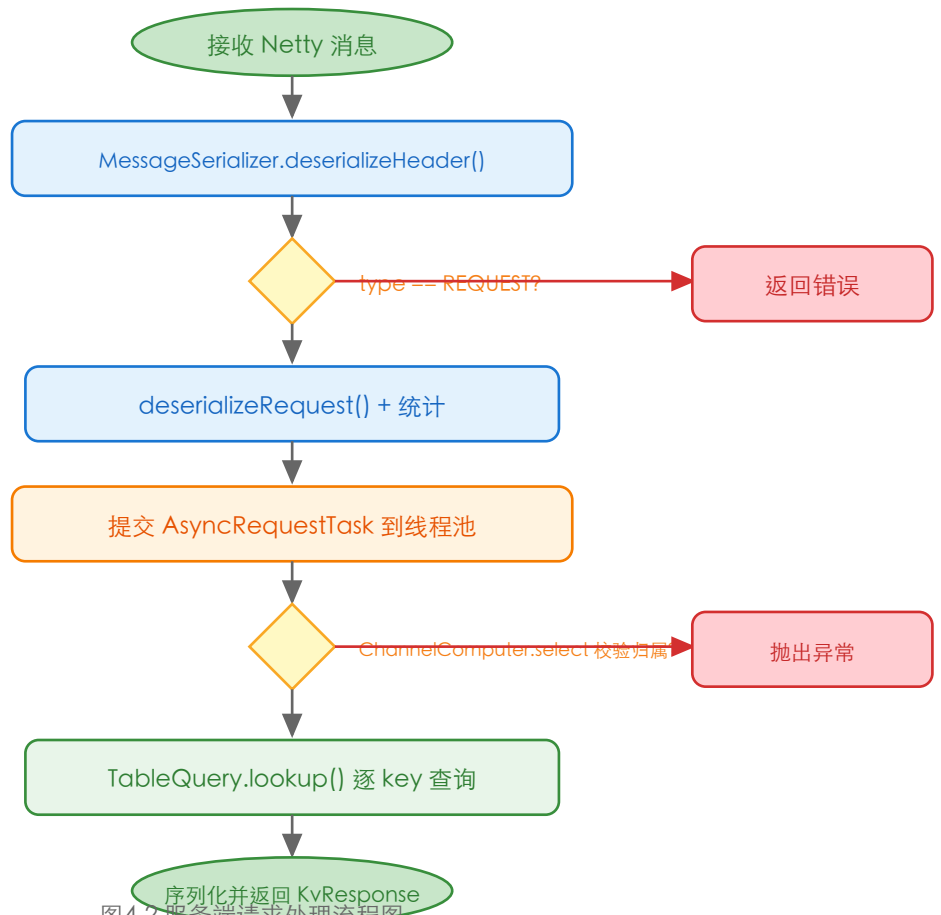


图 4-2 服务端请求处理流程图

4.3 服务器启动与端口绑定流程

`NetworkServer.start()` 遍历端口范围依次尝试绑定。每次绑定失败（`BindException`）时关闭当前 Netty 资源并尝试下一个端口，直到成功或端口耗尽。

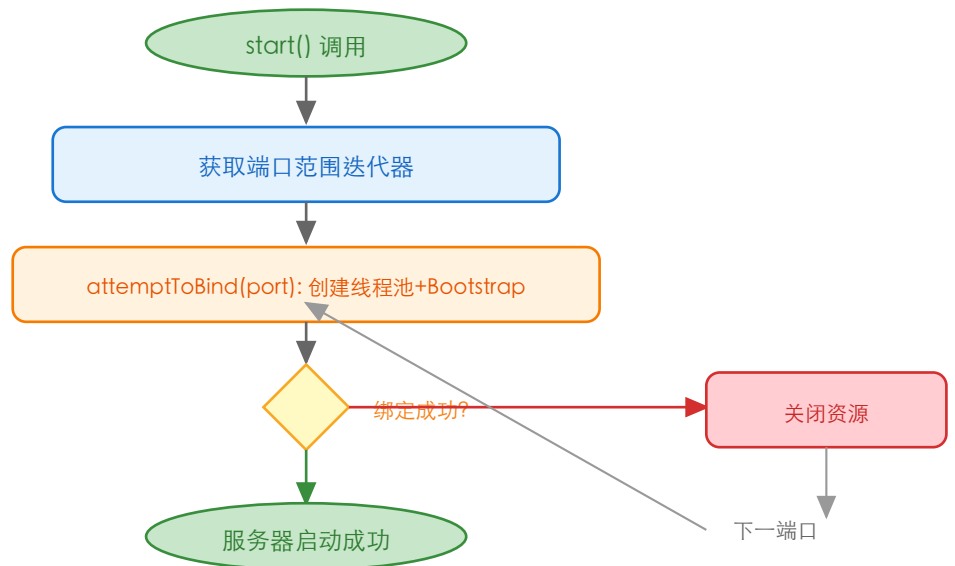


图4-3 服务器启动与端口绑定流程图

第5章 核心时序图

5.1 远程 KV 查询完整时序

以下时序图展示了一次完整的远程 KV 查询过程，从 KvQueryClient 发起请求到收到响应的全链路交互。

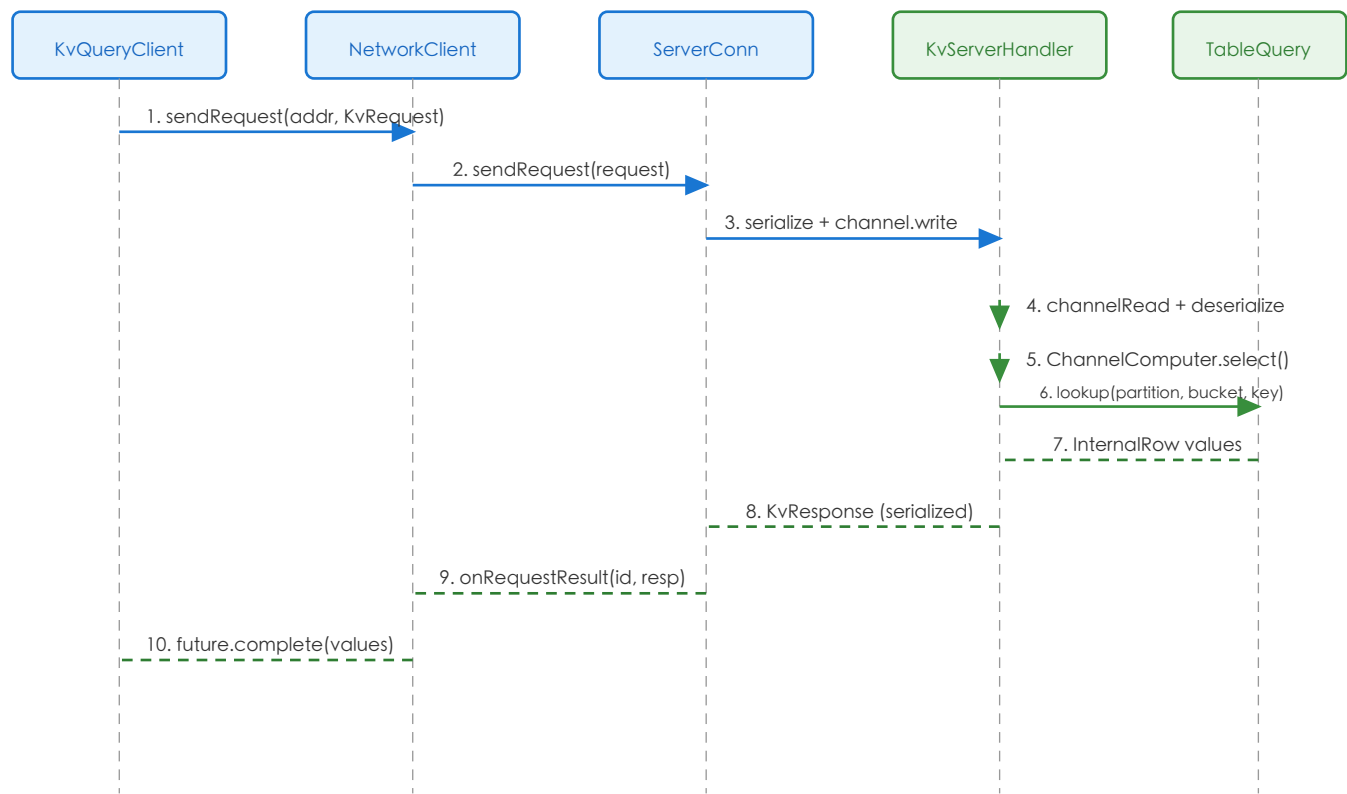


图5-1 远程 KV 查询完整时序图

5.2 失败重试时序图

当服务端抛出 `UnknownPartitionBucketException` 或客户端遇到 `ConnectException` 时，`KvQueryClient` 自动重试并强制刷新 location。

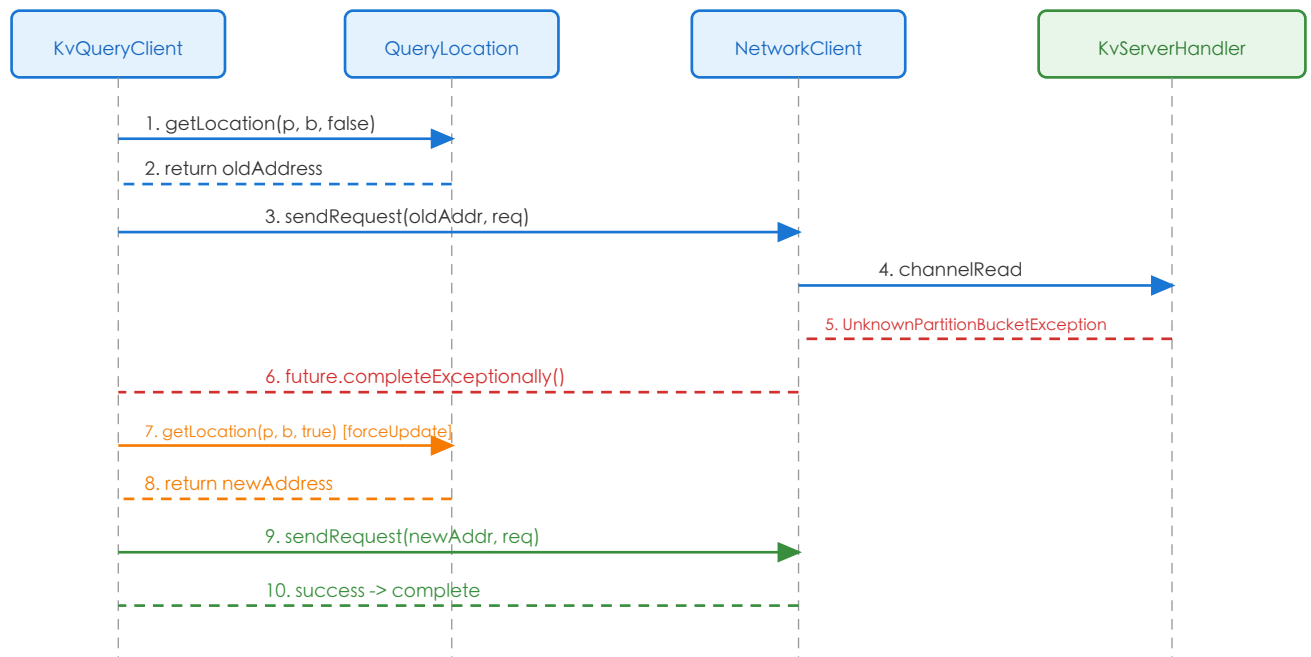


图5-2 失败重试时序图（`UnknownPartitionBucketException` 场景）

第6章 关键场景调用链

6.1 场景一：远程 KV 查询

触发条件：客户端调用 `KvQueryClient.getValues(partition, bucket, keys)`

调用链路：

步骤	类.方法	说明
1	<code>KvQueryClient.getValues()</code>	创建 <code>CompletableFuture</code> ，调用 <code>executeActionAsync</code>
2	<code>KvQueryClient.executeActionAsync()</code>	调用 <code>getResponse</code> 获取操作 <code>Future</code> ，注册回调
3	<code>KvQueryClient.getResponse()</code>	通过 <code>QueryLocation.getLocation()</code> 获取服务器地址
4	<code>NetworkClient.sendRequest(addr, req)</code>	获取或创建 <code>ServerConnection</code> ，发送请求
5	<code>ServerConnection.sendRequest()</code>	若 <code>Pending</code> 则排队，若 <code>Established</code> 则序列化写入 <code>Channel</code>
6	<code>EstablishedConnection.sendRequest()</code>	<code>MessageSerializer.serializeRequest()</code> 序列化，写入 <code>Channel</code>
7	<code>AbstractServerHandler.channelRead()</code>	服务端接收消息，反序列化并提交 <code>AsyncRequestTask</code>
8	<code>KvServerHandler.handleRequest()</code>	<code>ChannelComputer.select()</code> 校验归属，调用 <code>lookup</code>
9	<code>TableQuery.lookup(partition, bucket, key)</code>	本地查询 <code>key</code> 对应的 <code>value</code>
10	<code>AsyncRequestTask.run()</code> callback	序列化 <code>KvResponse</code> ，通过 <code>Channel</code> 写回客户端

6.2 场景二：服务器启动

触发条件：调用 `KvQueryServer.start()`

调用链路：

步骤	类.方法	说明
1	<code>KvQueryServer.start()</code>	调用父类 <code>NetworkServer.start()</code>
2	<code>NetworkServer.start()</code>	遍历端口范围，调用 <code>attemptToBind()</code>
3	<code>NetworkServer.attemptToBind(port)</code>	创建 <code>queryExecutor</code> 线程池
4	<code>KvQueryServer.initializeHandler()</code>	创建 <code>MessageSerializer</code> 和 <code>KvServerHandler</code>

步骤	类.方法	说明
5	NetworkServer.attemptToBind(port)	配置 ServerBootstrap (BufferPool、Watermark)
6	ServerBootstrap.bind().sync()	Netty 绑定端口，成功则记录 serverAddress

6.3 场景三：客户端连接建立

触发条件：首次向某服务器发送请求

调用链路：

步骤	类.方法	说明
1	NetworkClient.sendRequest()	检查 connections 映射，不存在则创建 PendingConnection
2	ServerConnection.createPendingConnection()	创建 PendingConnection 和 EstablishedConnection 工厂
3	Bootstrap.connect(addr, port)	Netty 发起 TCP 连接，注册 ChannelFutureListener
4	PendingConnection.sendRequest(req)	请求加入 queuedRequests 排队
5	PendingConnection.establishConnection()	连接成功，创建 EstablishedConnection
6	EstablishedConnection 构造函数	向 pipeline 添加 ClientHandler，上报统计
7	逐个转发排队请求	遍历 queuedRequests，调用 EstablishedConnection.sendRequest()

6.4 场景四：服务端优雅关闭

触发条件：调用 KvQueryServer.shutdown()

调用链路：

步骤	类.方法	说明
1	KvQueryServer.shutdown()	调用 shutdownServer().get(10s)
2	NetworkServer.shutdownServer()	CAS 设置 shutdownFuture
3	EventLoopGroup.shutdownGracefully()	关闭 Netty 事件循环组

步骤	类.方法	说明
4	KvServerHandler.shutdown()	返回已完成的 CompletableFuture（无额外资源）
5	ExecutorUtils.gracefulShutdown()	优雅关闭 queryExecutor 线程池（最长 10 分钟）
6	CompletableFuture.allOf()	等待以上三个 Future 全部完成

6.5 场景五：大响应分块传输

触发条件：响应大小超过 Channel 的 writeBufferHighWaterMark

调用链路：

步骤	类.方法	说明
1	AsyncRequestTask.run()	handleRequest() 返回 KvResponse
2	MessageSerializer.serializeResponse()	序列化响应为 ByteBuf
3	判断 serialResp.readableBytes()	与 highWaterMark (32KB) 比较
4	new ChunkedByteBuf(serialResp, highWatermark)	创建分块写入对象
5	ctx.writeAndFlush(ChunkedByteBuf)	配合 ChunkedWriteHandler 分块写入
6	ChunkedByteBuf.readChunk()	每次读取 chunkSize 字节，最后一块直接 slice

第7章 配置与扩展

7.1 关键配置参数

参数	默认值	说明
bindPortRange	由调用方传入	服务端端口范围，支持多端口自动探测
numEventLoopThreads	由调用方传入	Netty 事件循环线程数，影响网络 IO 吞吐
numQueryThreads	由调用方传入	查询执行线程池大小，影响并发查询处理能力
LOW_WATER_MARK	8KB	Netty 写缓冲低水位线，控制写入恢复时机
HIGH_WATER_MARK	32KB	Netty 写缓冲高水位线，超过时暂停写入
ChunkedByteBuf.chunkSize	= HIGH_WATER_MARK	大响应分块传输的块大小
PooledByteBufAllocator pageSize	8192	内存池页大小
PooledByteBufAllocator maxOrder	11	内存池最大阶数，16MB/chunk

7.2 扩展点

- 自定义消息类型：继承 `MessageBody` 并提供对应的 `MessageDeserializer` 实现，可扩展新的请求/响应类型
- 自定义服务端处理器：继承 `AbstractServerHandler` 实现 `handleRequest()` 和 `shutdown()`，可接入新的业务逻辑
- 自定义统计收集：实现 `ServiceRequestStats` 接口，可接入监控系统（如 Prometheus、Metrics）
- 自定义网络服务：继承 `NetworkServer` 实现 `initializeHandler()`，可构建新类型的查询服务

7.3 外部依赖

依赖	用途
paimon-shade-netty-4	网络通信框架（shaded 避免版本冲突）
paimon-shade-guava-30	ThreadFactoryBuilder 等并发工具
paimon-common	BinaryRow、序列化工具、Preconditions 等
paimon-core	TableQuery、QueryLocation、ServiceManager 等核心接口

第8章 设计亮点与总结

8.1 设计亮点

- 清晰的分层架构：网络传输层（NetworkClient/NetworkServer）与业务逻辑层（KvQueryClient/KvServerHandler）完全分离，网络框架可独立复用
- 优雅的连接状态机：ServerConnection 通过 PendingConnection/EstablishedConnection 状态切换，请求在连接建立前自动排队，连接就绪后批量转发，对上层完全透明
- 异步全链路设计：从客户端请求到服务端处理，全程基于 CompletableFuture 异步驱动，避免线程阻塞
- 端口范围自动探测：NetworkServer 支持多端口尝试绑定，提高部署灵活性和容错性
- 内存管理优化：NettyBufferPool 强制使用 Direct Buffer，配合 ChunkedByteBuffer 分块传输和高低水位线机制，有效控制内存使用
- 自动失败重试：KvQueryClient 在遇到 UnknownPartitionBucketException 或 ConnectException 时自动重试并强制刷新 location，提升查询可靠性
- 可插拔统计系统：ServiceRequestStats 接口配合 Disabled/Atomic 两种实现，支持按需开启/关闭统计

8.2 设计限制

- 重试策略简单：目前仅支持无限重试，缺少最大重试次数、退避策略（Backoff）等高级重试机制
- 无连接心跳：缺少 TCP 空闲连接心跳检测，长时间空闲的连接可能被中间设备断开
- 序列化不够高效：RequestFailure 和 ServerFailure 使用 Java 原生 ObjectOutputStream 序列化异常对象，性能和跨语言兼容性较差
- 单一查询模式：目前仅支持 KV 点查，不支持范围查询或批量扫描

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	四层架构清晰，职责明确，模块间通过接口隔离
设计模式运用	★★★★★	恰当运用模板方法、状态机、策略、装饰器、工厂方法等模式
异步设计	★★★★★	全链路 CompletableFuture 异步驱动，无阻塞调用
可扩展性	★★★★☆	核心框架可独立复用，但消息类型扩展需修改多处代码
容错能力	★★★★☆	自动重试和端口探测提供基础容错，但缺少高级重试策略
内存管理	★★★★★	Direct Buffer + 分块传输 + 水位线控制，设计精细
可观测性	★★★★☆	统计接口完善，但缺少分布式追踪集成

总体评价：paimon-service 是一个设计精良的分布式远程查询服务模块。基于 Netty 构建的网络框架具有优秀的抽象层次和可复用性，连接状态机设计优雅，异步全链路驱动确保了高吞吐量。该模块很好地服务于 Apache Paimon 主键表的远程 Lookup 场景，为分布式环境下的实时查询提供了可靠的基础设施。